

Project Report: miniDB - Concurrent Banking Database Engine

RAJ GANDHI (BT2024172)

1. Problem Statement

This project implements a multithreaded banking database system utilizing a client-server architecture. It supports multiple concurrent users with distinct access roles (admin, user, guest) to perform CRUD operations safely. The system is designed to guarantee data consistency and safe concurrent access by employing rigorous synchronization, file locking, and inter-process communication mechanisms.

2. Explanation of Required Concepts Implemented

4.1 Role-Based Authorization: Implemented by requiring clients to provide credentials upon connection, mapping them to 'admin', 'user', or 'guest' roles. The server's **is_authorized** function then intercepts every incoming command, verifying the requested action against the user's role permissions before execution, effectively restricting unauthorized operations like a guest attempting to drop a table.

4.2 File Locking: Safeguards database integrity at the file system level by applying POSIX `fcntl` locks directly to the underlying `.db` files. Write operations (`F_WRLCK`) request exclusive access, while read operations (`F_RDLCK`) utilize shared locks. This prevents data corruption if an external process attempts to read or modify the database files while the server is executing a CRUD operation.

4.3 Concurrency Control: Managed primarily through POSIX threads (pthreads), where the server spawns a detached thread for every accepted client connection. Because traditional POSIX file locks are process-oriented and do not block threads within the same process, a global `pthread_mutex_t` (`db_mutex`) is used to serialize database access, strictly preventing race conditions when multiple threads attempt to modify or read records simultaneously.

4.4 Data Consistency: Maintained by combining the thread-level mutexes and disk-level file locking to prevent race conditions, dirty reads, and lost updates.

4.5 Socket Programming: Forms the foundation of the client-server model, utilizing the `AF_INET` address family and `SOCK_STREAM` (TCP) for reliable communication. The server binds to a specified port and listens for connections, while clients connect and exchange formatted `Msg` structs containing commands and payloads, demonstrating robust communication between systems over a network.

4.6 Inter-Process Communication (IPC): Demonstrated through the implementation of an asynchronous logging system utilizing System V Message Queues. Client handler threads generate log messages (e.g., login events) and send them via `msgsnd` to a central queue, where a dedicated logger thread receives them via `msgrcv` and processes them independently, demonstrating effective IPC.

3. Screenshots / Output

```
raj@Raj:~/os_project2$ ./client

CONNECTED TO miniDB SERVER

SYSTEM LOGIN
Roles: [admin, user, guest]
Enter Role: admin
Enter Password: admin123
[+] Login successful!

AVAILABLE COMMANDS
SHOW TABLES           : List all tables
CREATE [table]          : New table (ADMIN)
DROP [table]            : Drop table (ADMIN)
INSERT [table] [acc_no] [name] [bal] : Add account
SELECT [table] *        : Retrieve all
SELECT [table] [acc_no] : Get account
UPDATE [table] [acc_no] [new_bal]    : Update balance
DELETE [table] [acc_no]              : Delete record
logout                             : End user session
exit                               : Disconnect fully

miniDB(admin) > create account
SUCCESS
Table created.

miniDB(admin) > insert account 1 Joe 2000
SUCCESS
Account added.

miniDB(admin) > insert account 2 Adam 3000
SUCCESS
Account added.

miniDB(admin) > update account 2 1000
SUCCESS
Balance updated.

miniDB(admin) > select account *
SUCCESS
  Acc: 1 | Name: Joe | Bal: $2000.00
  Acc: 2 | Name: Adam | Bal: $1000.00

miniDB(admin) > exit
Disconnecting. Goodbye!
```

```
raj@Raj:~/os_project2$ ./server

Banking Engine Listening...

[LOG] Client logged in as: admin
[LOG] Client disconnected.
^C
[server] Shutting down gracefully...
raj@Raj:~/os_project2$
```

4. Challenges Faced and Solutions

The most significant challenge was maintaining data consistency when multiple users attempted to update the same bank account simultaneously, which initially resulted in lost updates and race conditions. To resolve this, I implemented a global database mutex (`pthread_mutex_t`) combined with POSIX advisory file locking (`fcntl`), ensuring mutually exclusive access during writes and shared access during reads, which successfully prevented dirty reads. Another challenge was handling abrupt client disconnections that caused the server to crash due to broken pipes; this was solved by explicitly ignoring the `SIGPIPE` signal and implementing robust error checking on socket read/write operations to gracefully terminate the specific client thread without affecting the main server process.